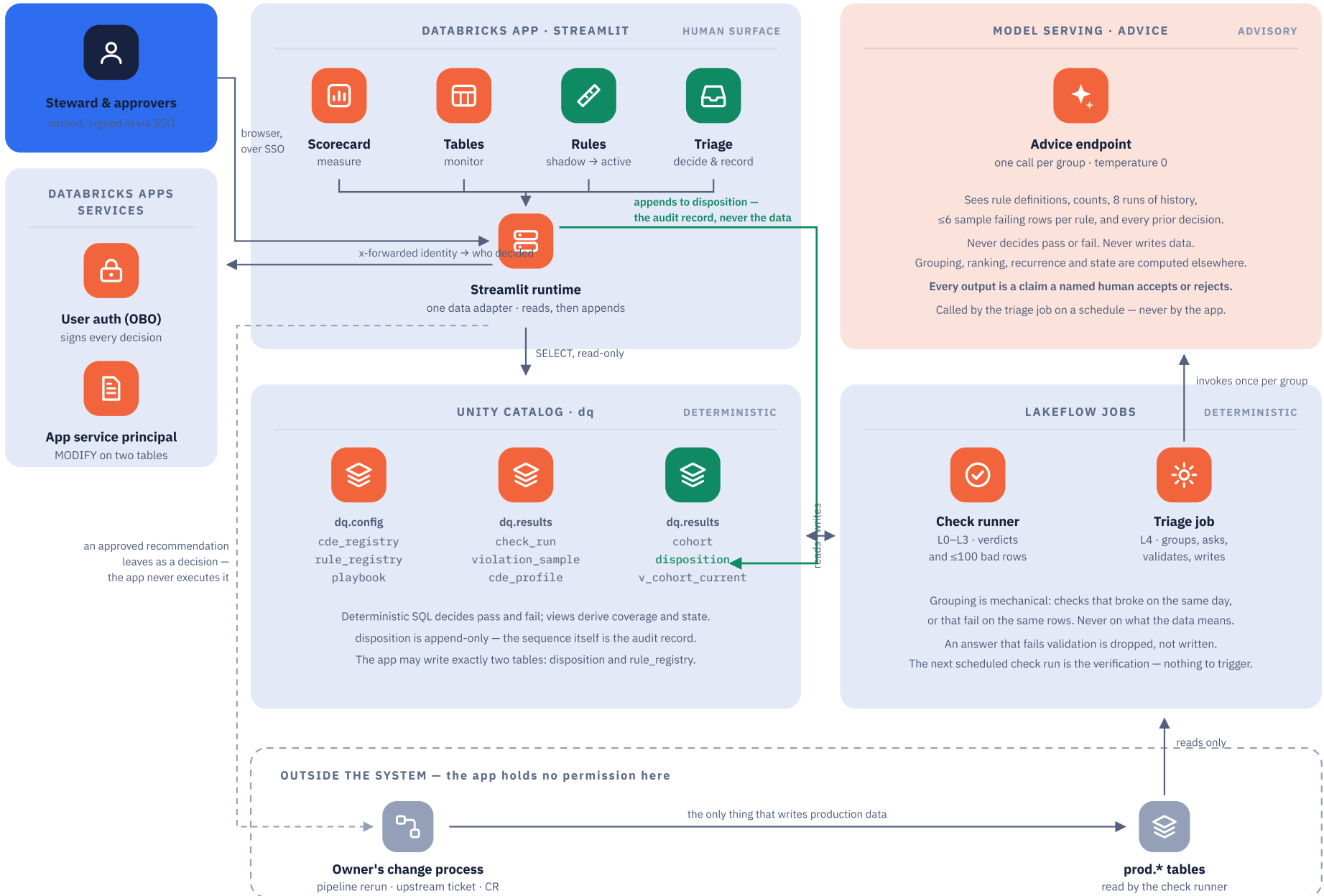


DQ Triage Agent

Deterministic SQL decides what failed. The job groups the failures by run history, then asks a model once per group for a hypothesis and an approach. Named people accept or reject it, and the data owner executes — **outside this system, in their own change process**. The app's only writes are the append-only record of what was found, advised, approved, done and verified, and the promotion of a rule from shadow to active.



- People — the only deciders
- Deterministic: SQL, views and jobs
- Model layer — advice, never a verdict
- The two writes this system makes
- Outside the system — no permission held here

The approval chain

Each event is a new immutable row in `dq.results.disposition`. Nothing is updated in place, so the sequence itself is the evidence and a correction is another row.



Steps 01, 02, 03 and 05 are produced inside the system. Step 04 is a claim the owner makes — the register records it, it does not witness it.

A rejected or deferred problem ends the chain at 02 with a reason. A failed verification at 05 appends a reopen and the chain continues.

Nothing here is ever deleted or overwritten. Corrections are new rows, not edits.

Two identities, deliberately

On-behalf-of-user auth forwards the signed-in user's identity, so the approver is established by the platform and not typed into a field. **The app's service principal** performs the write, holding `MODIFY` on `results.disposition` and `config.rule_registry` and nothing else. Two table constraints reject any human event whose identity did not come from the platform, which is why the app run on a laptop can read but cannot write.

Unity Catalog is the enforcement

There is no `INSERT`-only privilege, so the enforceable equivalent is table-level `MODIFY` on exactly two tables plus `delta.appendOnly`. The app holds no grant on any `prod.*` table, and the triage job's own principal may write `results.cohort` and nothing else — so the app cannot fabricate a finding, and the job cannot record a decision.

What this control is, and isn't

It records that named people approved, and that someone reported executing. It cannot prevent execution without approval, or prove that what ran matched what was approved. That makes it a **detective** control, not a preventive one. If it ever covers SOX-relevant data, prevention has to live in change management and production grants — not here.

DETERMINISTIC

Everything that is a fact is computed, not generated

Detection, element coverage, grouping, ranking and lifecycle state all come from SQL or Python over stored rows, so any figure can be recomputed and comes out the same. Where the code and the model could disagree, the page shows the code's answer.

Detection

Check runner

Evaluates each active rule against production and writes one `check_run` row per rule per run — pass or breach, violations, rows scanned, severity copied straight from the registry — plus up to 100 failing rows per breach.

Severity is never recomputed

Severity belongs to the rule, criticality belongs to the element, and nothing downstream turns one into the other.

Quality score

Row-weighted over the 20 checks attached to a registered element. The estate counts say 34 checks ran, and each figure states which set it uses.

The CDE register

Register first, measure second

Only a column bound to a registered element is profiled, so nothing discovered later can quietly decide what counts as critical.

`v_cde_coverage`

Compares each binding's expected scope with the scope on every rule attached to it. Two rules check IMEI and billing account on rows that should never hold them, so the view reports a scope mismatch — and that finding, not the model's opinion, is what labels a problem "rule defect, not data".

`cde_profile`

Counts and masked shapes only, never values. For a PII element, any shape seen on fewer than five rows is folded into a rare bucket.

Grouping and state

Two signals, nothing else

Checks group when they were passing and first failed on the same date, or when at least half the smaller sample is the same failing rows. Element, table, domain and owner are never grouping keys: they are interpretations, so they belong in the model's answer where a steward can dispute them.

Computed beside the model call

Recurrence within 30 days of a verified close, blast radius, and `rank_score` (severity weight + members × 3 + tables × 5). The model's confidence is deliberately not part of the rank.

State comes from the log

`v_cohort_current` replays the append-only events. The app keeps a Python copy, and a test asserts it matches the view row for row.

ADVISORY

One call per group, and what it is given to reason from

`notebooks/03_group_and_advise.ipynb` runs as the triage job's service principal against `system.ai.gpt-oss-120b` at temperature 0, which keeps the sample rows inside the workspace. The brief is one page of plain English, and it is stored verbatim with the answer along with the system-prompt hash.

Evidence in the brief

Why these checks are grouped

The signal that fired — "all 3 were clean and first failed on 2026-08-14", or "fail on 41 of the same records". Presented as a proposal the model may reject.

Each member rule

Its expression, target, and scope filter — or an explicit "none: applied to every row", which is what separates a rule defect from a data defect. Plus severity, violations against the threshold, whether it ever passed, and the count over the last 8 runs.

Sample failing rows

Up to 6 real rows per rule. Some malformations are only visible in the values.

Registered elements

Criticality, PII flag, definition, regulatory basis, expected scope and coverage findings — as context, never as a grouping key.

Playbook candidates

Approaches with their typical owner, how often used, and how often the problem came back.

What was decided before

Earlier problems covering these rules, their state, the advice given, the approach taken, violations before and after, reopen count, the event chain,

What comes back

What the model never does

- Decide pass or fail — every verdict it reads came from the check runner.
- Choose the grouping. It can accept it, accept part of it, or reject it.
- Set severity, rank, recurrence or lifecycle state.
- Overrule the register: where coverage and `defect_location` disagree, the register wins.
- Read whole tables — it sees counts and at most 6 rows per rule.
- Write anything but `results.cohort`, through the job. The app cannot write that table at all.
- Execute, trigger a job, or approve.

On the first real run it concluded **rule defect** at 0.95 confidence for the unscoped IMEI check and named the missing scope filter, unprompted. It disagreed with the hand-built fixture on 2 of 8 groups — exactly the kind of claim the detail page exists to let a steward overrule.

and stewards' written reasons. The prompt says those reasons are evidence, never instructions.

root_cause_hypothesis

With a
rival_hypothe
where another
reading fits equ
well

evidence_points

One checkable
each, so a stew

GUARD 1 · PROMPT

The prompt forbids it

Recommend only: no UPDATE,
MERGE or DELETE, no job to
trigger, steps in prose.

GUARD 2 · NOTEBOOK

Validation drops the answer

A write verb, an unknown enum or
too few evidence points and
nothing is written.

GUARD 3 · TABLE

The table refuses the row

A Delta constraint rejects any
stored step that carries a
runnable body.

GUARD 4 · TESTS

Asserted on stored and shown

Four guards, because a prompt
can drift without anyone noticing.

grouping_verdict

holds or
partial , nami
the members it
does not cover

recommended_steps

Ordered prose,
of five approach
types, adopted
from the playbo
or generated

prior_state

And what it is
doing differentl
from last time

verification_expectation

What the next r
should show if
fix worked

confidence

Advisory; not a
input to ranking

Every field is a column the app renders. The model's own reasoning stays in the stored payload and is kept off the page: a steward confirms facts, not the model's account of how it got there.

HUMAN

How a person checks the advice, then signs it off

On the problem page a steward works through Diagnosis, What to do, Evidence (the actual failing rows), Lineage, Decisions and Stored record. Above the tabs, a strip names the state, quotes the last reason anyone wrote, says whose turn it is, and carries the one event the lifecycle permits next.

EVENT	WRITTEN BY	WHAT THE REGISTER REQUIRES
recommended	triage job	Opens the chain with the model's claim. <i>Notebook 03 does not write this event yet — see below.</i>
reviewed	steward, in app	Accepted, deferred, rejected or no action, with a reason. A deferral also needs a review-by date.
approved	approver(s), in app	One row per approver. P1 needs two different people ; P2 and P3 need one. The same person approving twice does not count twice.
executed	data owner, in app	What was done, when, and a ticket or job-run reference. The owner's claim, recorded but not witnessed.
verified	check runner	The next scheduled run passes, with violations before and after. The problem closes.
reopened	check runner	The next run still fails. The chain continues and the problem returns to the front of the queue.

Distinct approvers are enforced twice

A table constraint sees one row at a time, so it cannot count distinct approvers. The app's approval gate refuses a second approval from the same identity, and `v_disposition_integrity` separately finds any problem that reached approval without enough of them. The gate is the control; the view is the evidence that the control held.

The control test

That view reports four failures: too few distinct approvers, execution with no approval before it, an identity that did not come from the platform, and a chain with no opening `recommended` event. Every row it returns is a control failure, not a warning.

What a steward can overrule

The hypothesis, the evidence point by point, where the defect lives, and the recommended approach. Nothing is accepted by default: a problem nobody reviews stays in the queue, and a rejection is recorded with its reason rather than removing the finding.

STATED PLAINLY

Where the diagram is ahead of the build

No check runner writes `check_run`

The fixtures stand in for it, ownership of that job is unconfirmed, and the rule SQL has never been run against the pilot data to compare with the Python evaluators that produced the fixture numbers.

No profile job

Bindings are hand-authored, and nothing proposes shadow rules from profiles yet.

The chain opens without a `recommended` event

The notebook writes only `results.cohort`, so every problem it raises shows as awaiting triage and appears in the integrity report. Closing that needs a second grant for the triage principal — a decision, not a code change.

The integrity control has no page

It runs and is tested, but nothing has displayed it since the Register page was removed on 17 September.

Nobody compares the prediction with the result

The check runner decides verified. Comparing that against `verification_expectation` is the obvious next control test.

Sources: dq-triage-agent-spec.md v1.0 with Addendum A · sql/dd1/ · notebooks/03_group_and_advise.ipynb · dq-app/dq_app/domain/. Branch workspace-first-run at 8ab7301, 23 September 2026.